

---

# 网络安全技术课程实验报告

## 实验一



实验名称：加解密网络聊天程序\_\_\_\_\_

姓名：\_\_\_\_段俊宇\_\_\_\_\_

学号：\_\_\_\_2313815\_\_\_\_\_

专业：\_\_\_\_信息安全、法学双学位班\_\_\_\_\_

提交日期：2026 年 3 月 22 日\_\_\_\_\_

# 一、实验目的

- 1. 掌握 SOCKET 编程并熟悉网络通信的过程。
- 2. 熟悉 DES、AES、ECC、RSA 等密码加密算法的加解密过程、不同模式和参数以及代码实现。
- 3. 实现聊天程序和加解密算法的结合，理解加密算法的完整性校验。

# 二、实验要求及要点

- 1. 熟悉多种密码加密、签名等算法的安全使用
- 2. 加密功能包括 DES、AES、RSA、ECC 等多种，了解不同算法、参数及加密模式的原理与差异。
- 3. 聊天过程中的消息进行签名和验签。
- 4. 支持 3 个及以上用户进行加密聊天，聊天内容消息中转服务器不可知，用户之间无法伪造身份和消息

# 三、实验内容

## 3.1 项目结构

在实验过程中我采用了模块化结构，将每种加密算法的代码放在一个单独的 python 文件中，并采用统一的函数接口便于调用。这些加密算法统一放在 Crypto\_Algorithm 文件夹中，作为加密算法模块。加密算法模块同目录下还有配置文件、服务端以及客户端，这样整体结构较为清晰，封装性好。

|                                                                                     |                  |                 |                       |      |
|-------------------------------------------------------------------------------------|------------------|-----------------|-----------------------|------|
|  | __pycache__      | 2026/3/21 22:51 | 文件夹                   |      |
|  | Crypto_Algorithm | 2026/3/21 16:44 | 文件夹                   |      |
|  | ecc_private_key  | 2026/3/21 22:49 | 文件夹                   |      |
|  | rsa_private_key  | 2026/3/21 22:51 | 文件夹                   |      |
|  | chat_client.py   | 2026/3/21 23:02 | JetBrains PyCharm ... | 4 KB |
|  | chat_server.py   | 2026/3/21 22:57 | JetBrains PyCharm ... | 4 KB |
|  | config.py        | 2026/3/21 22:50 | JetBrains PyCharm ... | 1 KB |
|  | README.md        | 2026/3/21 11:41 | MD 文件                 | 0 KB |
|  | requirements.txt | 2026/3/21 23:28 | Text 源文件              | 1 KB |

---

## 3.2 代码实现

接下来我将对各部分代码进行解释。

### 3.2.1 配置文件

本项目的配置文件为 config.py, 里面记录了 DES 和 AES 加密算法需要的密钥、签名所需的 HMAC 密钥、服务端 IP、端口以及默认加密算法。

```
1. SERVER_IP = "127.0.0.1"
2. SERVER_PORT = 9000
3. AES_KEY = b'nankaiuniversity'
4. DES_KEY = b'security'
5. HMAC_KEY = b'abcdef1234567890'
6.
7. # 默认加密方式
8. DEFAULT_ALGORITHM = "RSA"
```

### 3.2.2 DES 加解密算法

本部分内容基于模型生成的内容进行了改进。

```
1. from Crypto.Cipher import DES
2. from Crypto.Util.Padding import pad, unpad
3. from Crypto.Random import get_random_bytes
4. from Crypto.Hash import HMAC, SHA256
5. from base64 import *
6. import os
7. import sys
8.
9. sys.path.append(
10.     os.path.dirname(os.path.dirname(__file__))
11. )
12.
13. from config import DES_KEY, HMAC_KEY
14.
15. class DES_Crypto:
16.     # 初始化
17.     def __init__(self):
18.         self.key = DES_KEY
19.         self.hmac_key = HMAC_KEY
20.
21.     # 加密
22.     def encrypt(self, plaintext):
23.         iv = get_random_bytes(8)
24.         cipher = DES.new(self.key, DES.MODE_CBC, iv)
25.         ciphertext = cipher.encrypt(
```

```

26.         pad(plaintext.encode(), DES.block_size)
27.     )
28.     data = iv + ciphertext
29.
30.     return b64encode(data).decode()
31.
32.     # 解密
33.     def decrypt(self, cipherdata):
34.         data = b64decode(cipherdata)
35.         iv = data[:8]
36.         ciphertext = data[8:]
37.         cipher = DES.new(self.key, DES.MODE_CBC, iv)
38.         plaintext = unpad(
39.             cipher.decrypt(ciphertext),
40.             DES.block_size
41.         )
42.
43.         return plaintext.decode()
44.
45.     # 签名
46.     def sign(self, message):
47.         # 创建 HMAC 对象并计算 hmac 值
48.         h = HMAC.new(self.hmac_key, digestmod=SHA256)
49.         h.update(message.encode())
50.
51.         return b64encode(h.digest()).decode()
52.
53.     # 验证签名
54.     def verify(self, message, signature):
55.         # 重新计算 HMAC 值并比较
56.         h = HMAC.new(self.hmac_key, digestmod=SHA256)
57.         h.update(message.encode())
58.
59.         try:
60.             h.verify(b64decode(signature))
61.             return True
62.         except:
63.             return False

```

使用 DES 算法加密时，先从配置文件中读取 8 字节的密钥，然后调用封装库进行加密。我使用的是 CBC 模式，因此需要将向量和加密后的数据一起传输。为了方便传输，我使用 base64 对数据编码再进行传输。解密时先解码，从前 8 字节提取向量，然后使用密钥解密即可得到原始

---

消息。

签名函数中计算了原始消息的 HMAC 值,验证签名时只需要重新计算,再进行比对即可。

### 3.2.3 AES 加解密算法

本部分内容基于模型生成的内容进行了改进。

```
1. from Crypto.Cipher import AES
2. from Crypto.Random import get_random_bytes
3. from Crypto.Util.Padding import pad, unpad
4. from Crypto.Hash import HMAC, SHA256
5. import base64
6. import os
7. import sys
8.
9. sys.path.append(
10.     os.path.dirname(os.path.dirname(__file__))
11. )
12.
13. from config import AES_KEY, HMAC_KEY
14.
15. class AES_Crypto:
16.     # 初始化
17.     def __init__(self):
18.         self.key = AES_KEY
19.         self.hmac_key = HMAC_KEY
20.     # 加密
21.     def encrypt(self, plaintext):
22.         iv = get_random_bytes(16)
23.         cipher = AES.new(self.key, AES.MODE_CBC, iv)
24.         ciphertext = cipher.encrypt(
25.             pad(plaintext.encode(), AES.block_size)
26.         )
27.
28.         return base64.b64encode(iv + ciphertext).decode()
29.     # 解密
30.     def decrypt(self, cipherdata):
31.         data = base64.b64decode(cipherdata)
32.         iv = data[:16]
33.         ciphertext = data[16:]
34.         cipher = AES.new(self.key, AES.MODE_CBC, iv)
35.         plaintext = unpad(
36.             cipher.decrypt(ciphertext),
37.             AES.block_size
```



```
18.         # 不存在就生成
19.     else:
20.         self.private_key = ec.generate_private_key(ec.SECP256R1())
21.         with open(self.key_file, "wb") as f:
22.             f.write(
23.                 self.private_key.private_bytes(
24.                     encoding=serialization.Encoding.PEM,
25.                     format=serialization.PrivateFormat.TraditionalOpenSSL,
26.                     encryption_algorithm=serialization.NoEncryption()
27.                 )
28.             )
29.         self.public_key = self.private_key.public_key()
30.
31.     # 导出公钥至服务端
32.     def export_public_key(self):
33.         return self.public_key.public_bytes(
34.             encoding=serialization.Encoding.PEM,
35.             format=serialization.PublicFormat.SubjectPublicKeyInfo
36.         ).decode()
37.
38.     # 加密
39.     def encrypt(self, plaintext):
40.         # ECC 一般不直接加密消息
41.         return plaintext
42.     # 解密
43.     def decrypt(self, ciphertext):
44.         return ciphertext
45.     # 签名
46.     def sign(self, message):
47.         signature = self.private_key.sign(
48.             message.encode(),
49.             ec.ECDSA(hashes.SHA256())
50.         )
51.
52.         return b64encode(signature).decode()
53.     # 验证签名
54.     def verify(self, message, signature, sender_public_key=None):
55.         if sender_public_key:
56.             # 如果公钥是字符串（PEM 格式），将字符串编码为字节，然后加载为公钥对象
57.             if isinstance(sender_public_key, str):
58.                 pub_key =
serialization.load_pem_public_key(sender_public_key.encode())
59.             else:
60.                 pub_key = sender_public_key
```

```
61.         # 验证
62.         try:
63.             pub_key.verify(b64decode(signature), message.encode(),
ec.ECDSA(hashes.SHA256()))
64.             return True
65.         except:
66.             return False
```

ECC 加解密算法属于公钥体制算法，通过密钥对消息签名，另一方验证签名即可得到消息完整性，其安全性基于离散对数难题。在签名时，发送方先对消息计算哈希值，再使用自己的私钥对哈希值签名。验证签名时，使用同样方法计算，如果两次计算的结果相同则有效。为了保证安全，每个用户都会有一对独一无二的公钥和私钥，在代码中，私钥保存在本地，而公钥上传至服务端保存。

一般来说在通信中使用 ECC+AES 混合加密，ECC 用于保护消息完整性，AES 用于对消息内容进行加密。我的代码实现主要实现了 ECC 签名的过程，加解密未结合其他算法。但是在下面的 RSA 部分实现了消息机密性和完整性的保护。

### 3.2.5 RSA 加解密算法

本部分内容基于模型生成的内容进行了改进。

```
1. import os
2. from Crypto.PublicKey import RSA
3. from Crypto.Cipher import PKCS1_OAEP
4. from Crypto.Signature import pkcs1_15
5. from Crypto.Hash import SHA256
6. from base64 import b64encode, b64decode
7.
8. class RSA_Crypto:
9.     # 初始化
10.    def __init__(self, username, key_size=2048):
11.        os.makedirs("rsa_private_key", exist_ok=True)
12.        self.key_file = os.path.join("rsa_private_key",
f"{username}_rsa_private.pem")
13.        # 如果私钥存在就读取
14.        if os.path.exists(self.key_file):
15.            with open(self.key_file, "rb") as f:
16.                self.private_key = RSA.import_key(f.read())
17.        # 不存在就生成
18.        else:
```

```
19.         key = RSA.generate(key_size)
20.         self.private_key = key
21.         with open(self.key_file, "wb") as f:
22.             f.write(key.export_key())
23.         self.public_key = self.private_key.publickey()
24.
25.     # 导出公钥
26.     def export_public_key(self):
27.         return self.public_key.export_key().decode()
28.
29.     # 加密
30.     def encrypt(self, plaintext, receiver_public_key=None):
31.         pub_key = RSA.import_key(receiver_public_key.encode())
32.         cipher = PKCS1_OAEP.new(pub_key)
33.         ciphertext = cipher.encrypt(plaintext.encode())
34.
35.         return b64encode(ciphertext).decode()
36.
37.     # 解密
38.     def decrypt(self, ciphertext):
39.         cipher = PKCS1_OAEP.new(self.private_key)
40.         plaintext = cipher.decrypt(
41.             b64decode(ciphertext)
42.         )
43.
44.         return plaintext.decode()
45.
46.     # 签名
47.     def sign(self, message):
48.         h = SHA256.new(message.encode())
49.         signature = pkcs1_15.new(self.private_key).sign(h)
50.
51.         return b64encode(signature).decode()
52.
53.     # 验证签名
54.     def verify(self, message, signature, sender_public_key=None):
55.         try:
56.             if sender_public_key:
57.                 pub_key = RSA.import_key(sender_public_key.encode())
58.             else:
59.                 pub_key = self.public_key
60.             h = SHA256.new(message.encode())
61.             pkcs1_15.new(pub_key).verify(
62.                 h,
```

```
63.         b64decode(signature)
64.     )
65.     return True
66. except:
67.     return False
```

RSA 加解密算法也属于公钥体制算法，其安全性基于大整数的分解问题，目前认为模数在 2048bit 及以上才算安全。在我的代码中，同样为每个用户创建了一对独一无二的公钥和私钥，私钥保存在本地，公钥上传至服务端。在加密时，发送方使用自己的私钥加密，然后使用对方的公钥签名；接收方先使用自己的私钥验证签名，然后使用对方的公钥解密，确保了消息的完整性和内容的机密性。

RSA 相对于 ECC 具有计算开销大的缺点，同时它无法一次性传送大量数据，需要分块进行传送，因此通常用于签名而不是加密。

### 3.2.6 服务端实现

本部分内容基于模型生成的内容进行了改进。

```
1. import socket
2. from threading import Thread
3. import json
4.
5. # 存储客户端及映射关系 username -> {"conn": socket, "public_key": str}
6. users = {}
7.
8. # 消息转发
9. def forward_message(data):
10.     try:
11.         # 提取发送方和接收方
12.         msg = json.loads(data.decode())
13.         receiver = msg["to"]
14.
15.         if receiver in users:
16.             # 需要获取 conn 对象
17.             users[receiver]["conn"].send(data)
18.     except:
19.         pass
20.
21. # 客户端处理
22. def handle_client(conn):
23.     username = None
24.     try:
```

```
25.     # 第一次发送消息时注册用户名
26.     data = conn.recv(4096)
27.     msg = json.loads(data.decode())
28.
29.     if msg["type"] == "register":
30.         username = msg["username"]
31.         public_key = msg.get("public_key")
32.         users[username] = {"conn": conn, "public_key": public_key}
33.         print(f"{username} connected")
34.
35.     # 消息处理循环
36.     while True:
37.         data = conn.recv(4096)
38.         if not data:
39.             break
40.
41.         msg = json.loads(data.decode())
42.
43.         # 判断消息类型
44.         if msg["type"] == "message":
45.             target = msg["to"]
46.             sender = msg["from"]
47.             if target in users:
48.                 # 转发消息并附加发送者公钥
49.                 if users[sender]["public_key"]:
50.                     msg["sender_pub_key"] = users[sender]["public_key"]
51.                     users[target]["conn"].send(json.dumps(msg).encode())
52.             # 请求消息，发送对应的公钥
53.             elif msg["type"] == "get_pubkey":
54.                 target = msg["username"]
55.                 if target in users:
56.                     response = {
57.                         "type": "pubkey",
58.                         "public_key": users[target]["public_key"]
59.                     }
60.                     conn.send(json.dumps(response).encode())
61.             else:
62.                 # 其他类型消息直接转发
63.                 forward_message(data)
64.
65.     except Exception as e:
66.         print(f"处理错误: {e}")
67.
68.     # 用户断开
```

```

69.     conn.close()
70.     for name, info in list(users.items()):
71.         if info["conn"] == conn:
72.             del users[name]
73.             print(f"{username} disconnected")
74.
75. # 服务端启动
76. def start():
77.     # 创建 SOCKET
78.     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
79.     # 绑定地址
80.     server.bind(("0.0.0.0", 9000))
81.     # 监听
82.     server.listen()
83.
84.     print("Secure Chat Server Started")
85.     # 建立连接循环
86.     while True:
87.         # 阻塞等待客户端连接
88.         conn, addr = server.accept()
89.
90.         print("New connection:", addr)
91.         # 多线程
92.         thread = Thread(
93.             target=handle_client,
94.             args=(conn,),
95.             daemon=True
96.         )
97.         thread.start()
98.
99. if __name__ == "__main__":
100.     try:
101.         start()
102.     except KeyboardInterrupt:
103.         print("服务器已停止")
104.

```

上面是服务端的实现代码，每个客户端在连接服务端后会单独创建一个线程，避免冲突。用户注册时会向服务端发送 register 类型的消息，其中包含用户名，如果使用 ECC 或 RSA 加密算法还需要上传公钥，服务端会将其加入字典中便于之后查找，之后在服务端窗口中输出 xxx connected，客户端关闭时输出 xxx disconnected。

在使用 RSA 加密时，需要接收方的公钥进行签名，因此客户端会发送 request 类型的消息，服务端会返回对应用户的公钥。对于其他的正常消息，服务端只负责转发，而没有对消息内容进行解密和查看，保障用户隐私不被泄露。

### 3.2.7 客户端实现

本部分内容基于模型生成的内容进行了改进。

```
1. import socket
2. from threading import Thread
3. import json
4. from config import DEFAULT_ALGORITHM, SERVER_IP, SERVER_PORT
5. from Crypto_Algorithm import get_crypto
6. from datetime import datetime
7. import queue
8.
9. class Colors:
10.     GREEN = '\033[92m'      # 绿色 - 发送的消息
11.     BLUE = '\033[94m'       # 蓝色 - 接收的消息
12.     RED = '\033[91m'        # 红色 - 错误信息
13.     END = '\033[0m'         # 重置颜色
14.
15. # 选择加密算法
16. algo = DEFAULT_ALGORITHM
17. # 输入用户名
18. username = input("你的用户名: ")
19. # 初始化加密模块
20. crypto = get_crypto(algo, username)
21. client = socket.socket()
22. client.connect((SERVER_IP, SERVER_PORT))
23. # 注册信息
24. register_data = {
25.     "type": "register",
26.     "username": username
27. }
28. if algo.upper() in ["RSA", "ECC"]:
29.     register_data["public_key"] = crypto.export_public_key()
30.
31. client.send(json.dumps(register_data).encode())
32. print(f"{Colors.GREEN}✓ 登录成功! {Colors.END}\n")
33.
34. pubkey_responses = queue.Queue()
35.
```

```
36. # 获取当前时间戳
37. def get_timestamp():
38.     return datetime.now().strftime("%H:%M:%S")
39.
40. # 接收信息
41. def receive():
42.     while True:
43.         try:
44.             msg = client.recv(4096)
45.             if not msg:
46.                 break
47.             data = json.loads(msg.decode())
48.             if data.get("type") == "pubkey":
49.                 pubkey_responses.put(data)
50.             elif data.get("type") == "message":
51.                 # 提取信息
52.                 sender = data["from"]
53.                 ciphertext = data["ciphertext"]
54.                 signature = data.get("signature")
55.                 sender_pub = data.get("sender_pub_key")
56.                 # 只有非对称算法才需要验证签名
57.                 if algo.upper() in ["RSA", "ECC"] and signature:
58.                     if not crypto.verify(ciphertext, signature,
sender_public_key=sender_pub):
59.                         print(f"{Colors.RED}{sender}: 签名验证失败{Colors.END}")
60.                         continue
61.                     plaintext = crypto.decrypt(ciphertext)
62.                     print(f"{Colors.BLUE}[{get_timestamp()}]{sender}:
{plaintext}{Colors.END}")
63.                     print(f"{Colors.GREEN}", end="", flush=True)
64.         except Exception as e:
65.             print("接收异常:", e)
66.             break
67.
68. # 发送消息
69. def send():
70.     while True:
71.         print(f"{Colors.GREEN}", end="") # 设置颜色
72.         text = input() # 输入的文字会变成绿色
73.         print(f"{Colors.END}", end="") # 重置颜色
74.         if ":" not in text:
75.             print(f"{Colors.RED}格式: 用户名:消息{Colors.END}")
76.             continue
77.         target, message = text.split(":", 1)
```

```

78.         if algo.upper() == "RSA":
79.             # 向服务端请求密钥
80.             request = {
81.                 "type": "get_pubkey",
82.                 "username": target
83.             }
84.             client.send(json.dumps(request).encode())
85.             # 等服务器返回
86.             try:
87.                 resp_data = pubkey_responses.get(timeout=5) # 等待最多 5 秒
88.             except queue.Empty:
89.                 print("获取公钥超时")
90.                 continue
91.             receiver_pub = resp_data["public_key"]
92.             ciphertext = crypto.encrypt(message, receiver_pub)
93.         else:
94.             ciphertext = crypto.encrypt(message)
95.         # 拼接格式
96.         data = {
97.             "type": "message",
98.             "from": username,
99.             "to": target,
100.            "ciphertext": ciphertext
101.        }
102.
103.        # 非对称算法加签名
104.        if algo.upper() in ["RSA", "ECC"]:
105.            signature = crypto.sign(ciphertext)
106.            data["signature"] = signature
107.
108.        client.send(json.dumps(data).encode())
109.
110.    if __name__ == "__main__":
111.        Thread(target=receive, daemon=True).start()
112.        send()

```

上面是客户端的实现代码，使用后台线程监听并接收消息，而发送消息函数在前台运行。客户端根据配置文件读取默认加密方式并初始化，在用户输入用户名后发送注册信息，开始正式聊天。

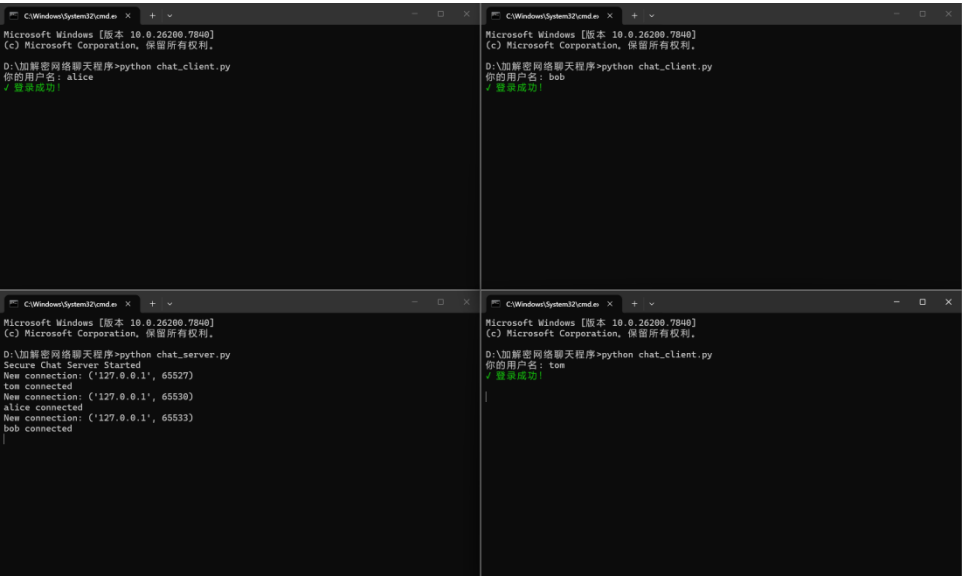
在发送消息时，先进行加密，如果不是公钥加密算法则不需要签名，使用 RSA 算法需要向服务端请求公钥进行签名。在接收消息时，如果属于公钥相应类型的消息则放入队列中，然后让发送函数进行处理，这样

可以避免线程冲突。对于其他类型的消息，先从 json 格式中提取信息，如果不是公钥加密算法则直接解密，否则需要验证签名，之后在窗口中显示时间、发送方和消息内容。

由于我实现的是端到端加密通信，因此需要输入接收方和消息内容。为了区分发送的消息和接收的消息，我定义了颜色类，发送的消息为绿色，接收的消息为蓝色，系统错误为红色，便于区分。

### 3.3 结果展示

下面我将 RSA 为例演示我的代码。



```
Microsoft Windows [版本 10.0.26280.7840]
(C) Microsoft Corporation. 保留所有权利。

D:\加密网络聊天程序>python chat_client.py
你的用户名: alic
√ 登录成功!
```

```
Microsoft Windows [版本 10.0.26280.7840]
(C) Microsoft Corporation. 保留所有权利。

D:\加密网络聊天程序>python chat_client.py
你的用户名: bob
√ 登录成功!
```

```
Microsoft Windows [版本 10.0.26280.7840]
(C) Microsoft Corporation. 保留所有权利。

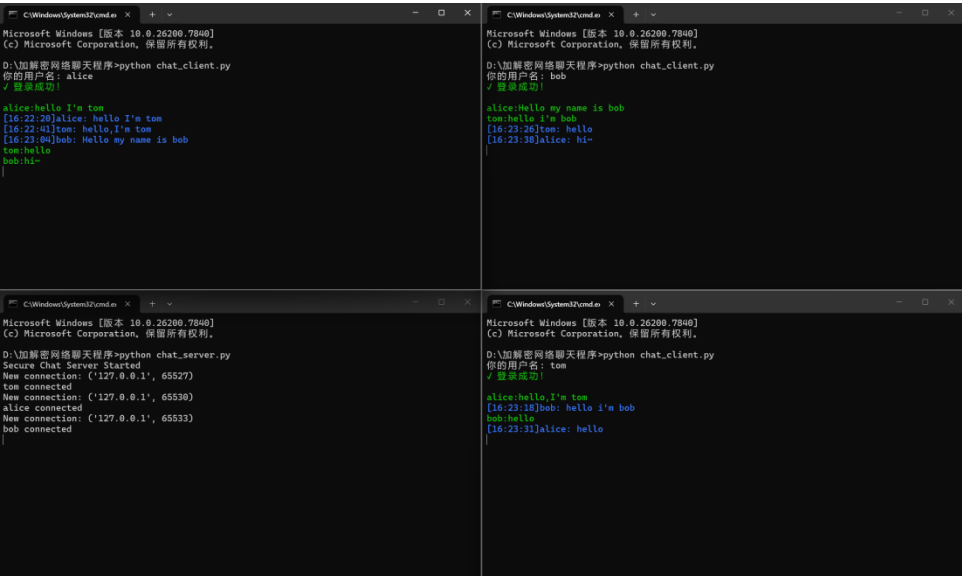
D:\加密网络聊天程序>python chat_server.py
Secure Chat Server Started
New connection: ('127.0.0.1', 65527)
tom connected
New connection: ('127.0.0.1', 65530)
alice connected
New connection: ('127.0.0.1', 65533)
bob connected
```

```
Microsoft Windows [版本 10.0.26280.7840]
(C) Microsoft Corporation. 保留所有权利。

D:\加密网络聊天程序>python chat_client.py
你的用户名: tom
√ 登录成功!
```

如上所示，代码能够正常运行，左下角是服务端，其他三个是客户端。

下图中三个客户端互相发消息可以收到，说明算法的加解密和签名验证过程没有问题。



```
Microsoft Windows [版本 10.0.26280.7840]
(C) Microsoft Corporation. 保留所有权利。

D:\加密网络聊天程序>python chat_client.py
你的用户名: alic
√ 登录成功!

alice:hello i'm tom
[16:22:20]alice: hello i'm tom
[16:22:41]tom: hello,i'm tom
[16:23:04]bob: Hello my name is bob
tom:hello
bob:hi-
```

```
Microsoft Windows [版本 10.0.26280.7840]
(C) Microsoft Corporation. 保留所有权利。

D:\加密网络聊天程序>python chat_client.py
你的用户名: bob
√ 登录成功!

alice:Hello my name is bob
tom:hello i'm bob
[16:23:26]tom: hello
[16:23:30]alice: hi-
```

```
Microsoft Windows [版本 10.0.26280.7840]
(C) Microsoft Corporation. 保留所有权利。

D:\加密网络聊天程序>python chat_server.py
Secure Chat Server Started
New connection: ('127.0.0.1', 65527)
tom connected
New connection: ('127.0.0.1', 65530)
alice connected
New connection: ('127.0.0.1', 65533)
bob connected
```

```
Microsoft Windows [版本 10.0.26280.7840]
(C) Microsoft Corporation. 保留所有权利。

D:\加密网络聊天程序>python chat_client.py
你的用户名: tom
√ 登录成功!

alice:hello,i'm tom
[16:23:18]bob: hello i'm bob
bob:hello
[16:23:31]alice: hello
```

---

其他三种算法的运行结果和上面相同，这里就不再重复展示，由此成功将 DES、AES、ECC、RSA 四种加密算法应用在网络聊天程序中，完成了实验内容。

## 四、关于人工智能使用的说明

### 4.1 人工智能使用声明

本人段俊宇承诺：对此文档的所有内容正确性、真实性负责，充分理解了提交报告的知识内容，并对使用大模型辅助的内容进行了正确的标注。

### 4.2 人工智能使用标注

在本次实验中，我的实验代码由大模型生成，通过询问大模型加深对代码的理解，在此基础上我对生成的内容进行调试、修改最终得到了实验报告中的代码。整个实验报告的代码解释及其他内容由我自己独立完成，完全基于我对代码的理解，没有使用大模型。

### 4.3 人工智能技术使用总结

我使用 Chatgpt 和 deepseek 辅助我完成本次实验，使用过程中，我发现 deepseek 的生成速度和 Chatgpt 相比较慢，并且会给代码加很多冗余性的防护，让代码看起来行数很多，很臃肿；Chatgpt 生成的代码较为简洁，并且对于修改内容显示较为清晰，但是 Chatgpt 的免费额度有限，不能无限使用。

因此，我认为 Chatgpt 适合生成代码并指出存在的问题，deepseek 由于免费，更适合代码解释和问题解答，便于加深对代码内容的理解。